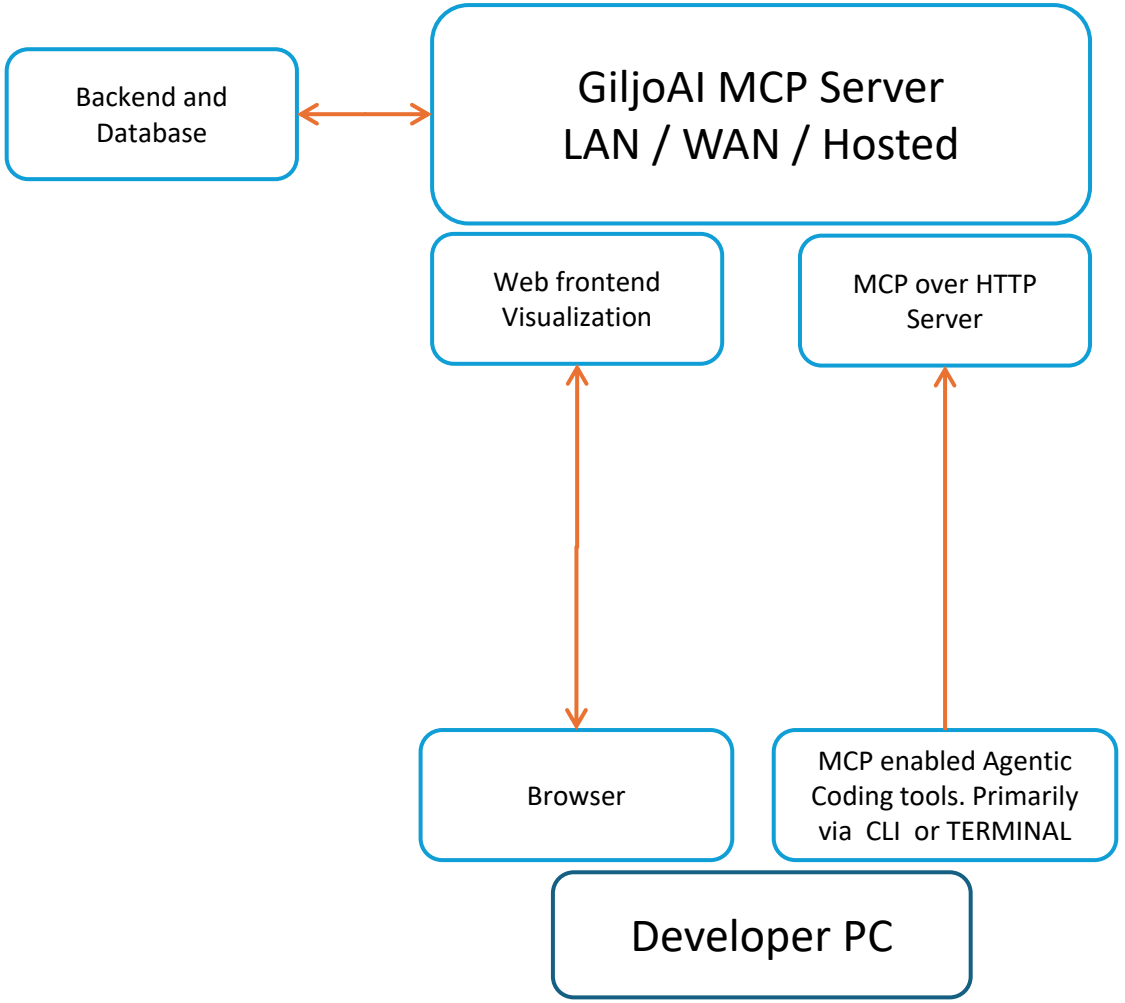
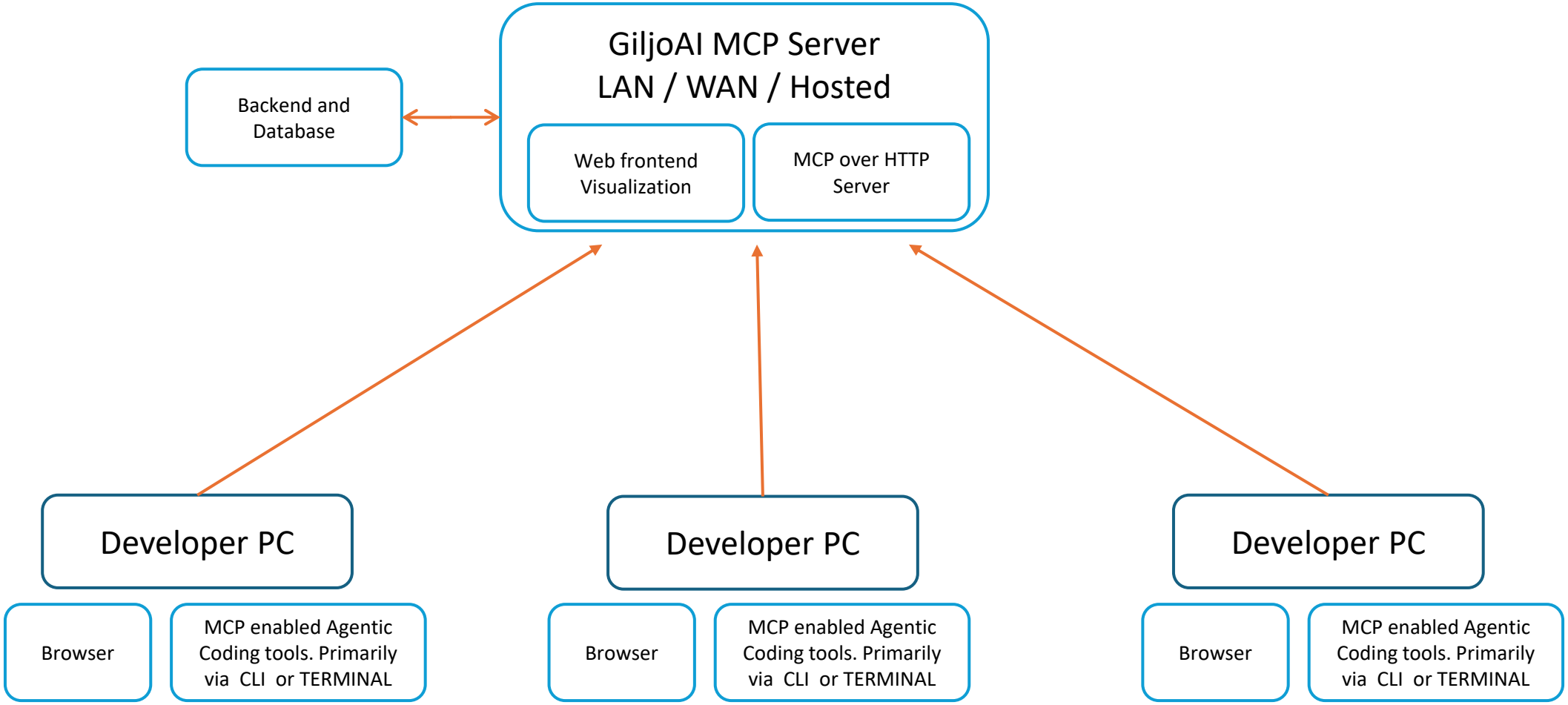
GiljoAI MCP Server

Workflows

Overall operational architecture



Multiuser architecture



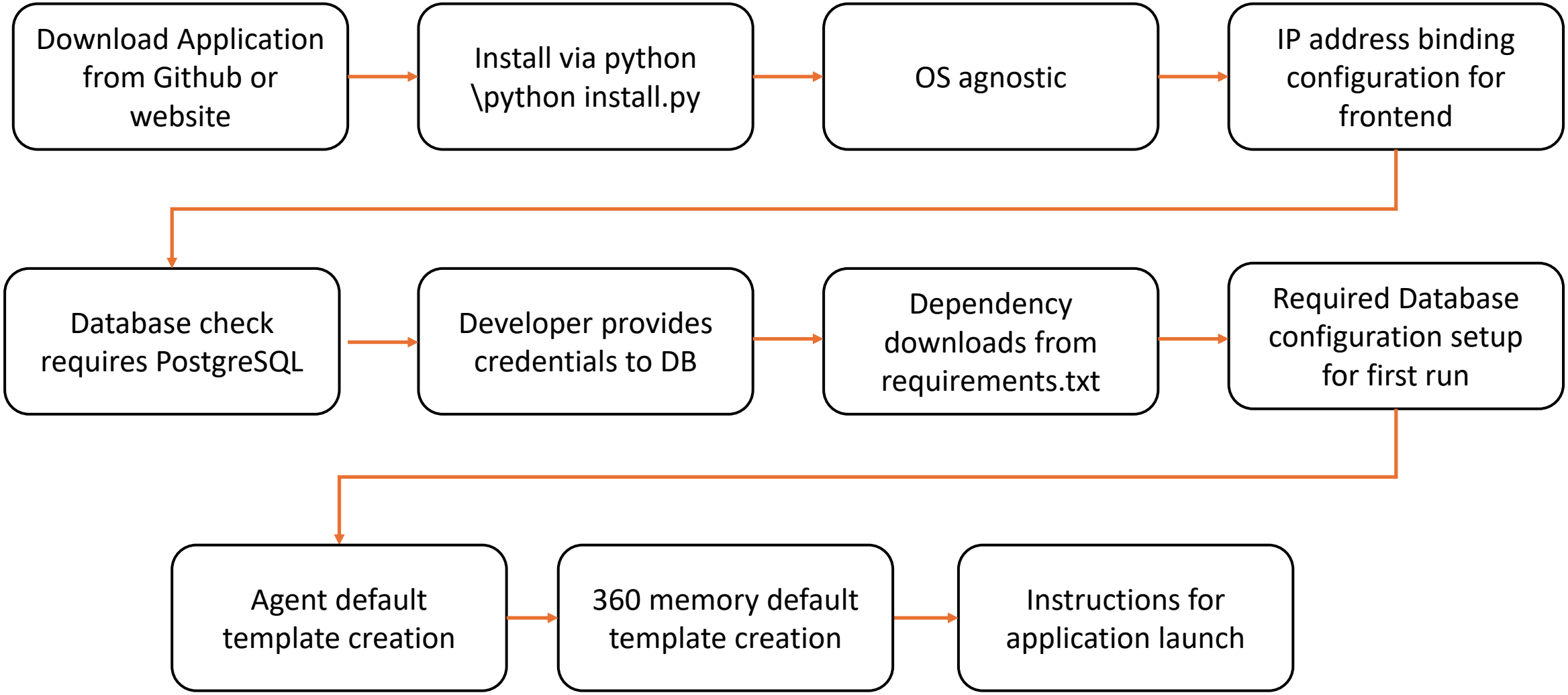
Server application layers

- OS agnostic Server
- Public IP based application interaction, not localhost for users
- Localhost interaction only for system functions between Backend, Frontend and PostgreSQL
- MCP over HTTP, not STDIO
- PostgreSQL Database
- Single Org architecture
- Tenant management (Users, with an Admin)
- API key and bearer key tenant separation
- Task, Product and Project management
- Jobs management

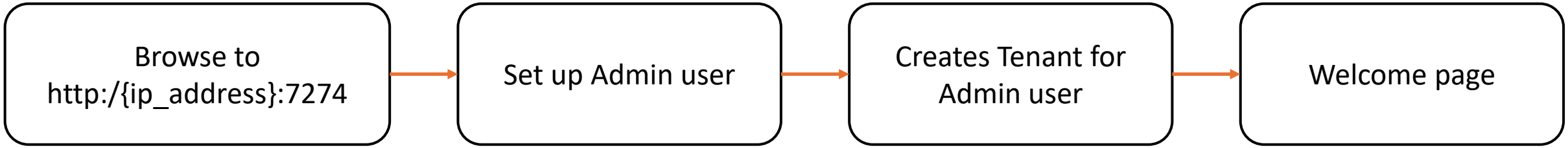
GiljoAI MCP Server
LAN / WAN / Hosted

- MCP integration setup tools (Simplified Integration to CLI/Terminal Agentic coding tools like Claude CLI, Codex CLI and Gemini CLI
- Slash Command Setup tools (Simplified Integration to CLI/Terminal Agentic coding tools like Claude CLI, Codex CLI and Gemini CLI
- Agent Template import setup tools(Simplified Integration to Claude CLI subagent feature)
- Agent Template management, creation and activation/enablement
- Serena MCP integration, a method of prompt modification of our static prompts for “Orchestrator agent”, and context to add to jobs and missions for Agents as they get assigned tasks.
- Github integration
- MCP message center for working Agents, with Developer visualization
- Product , Project and Jobs context management with priority assignment
- 360 memory management, mini git, lifecycle, github tracking

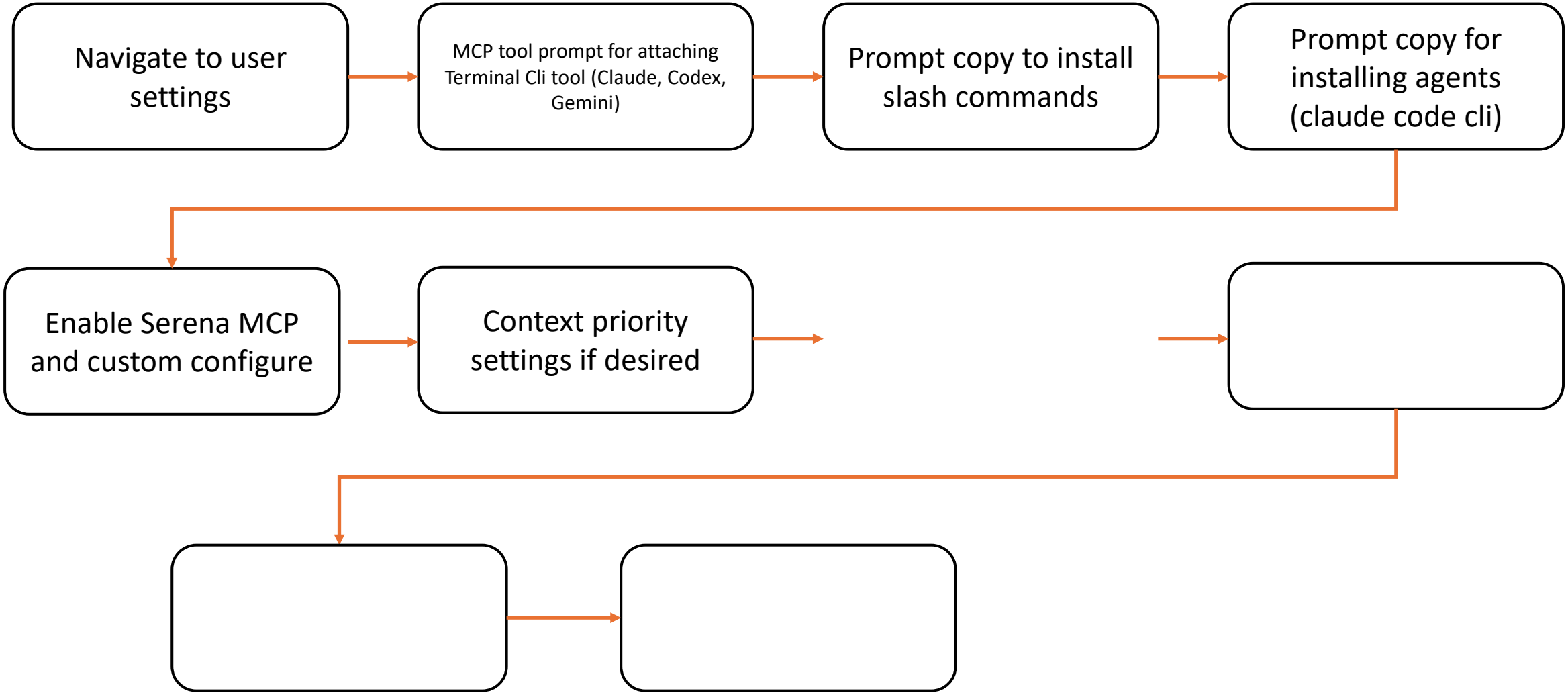
Installation and Setup



First run workflow

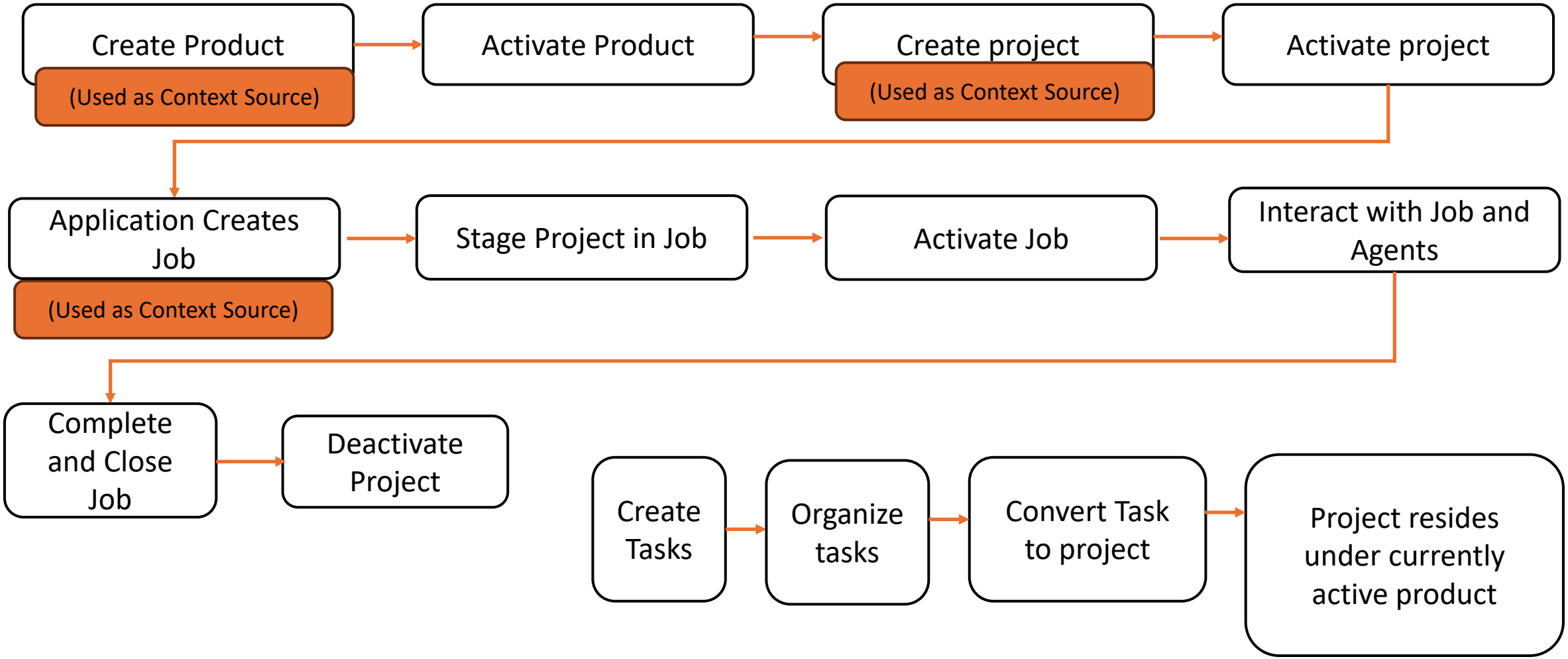


Application Setup overview



Application Workflow overview

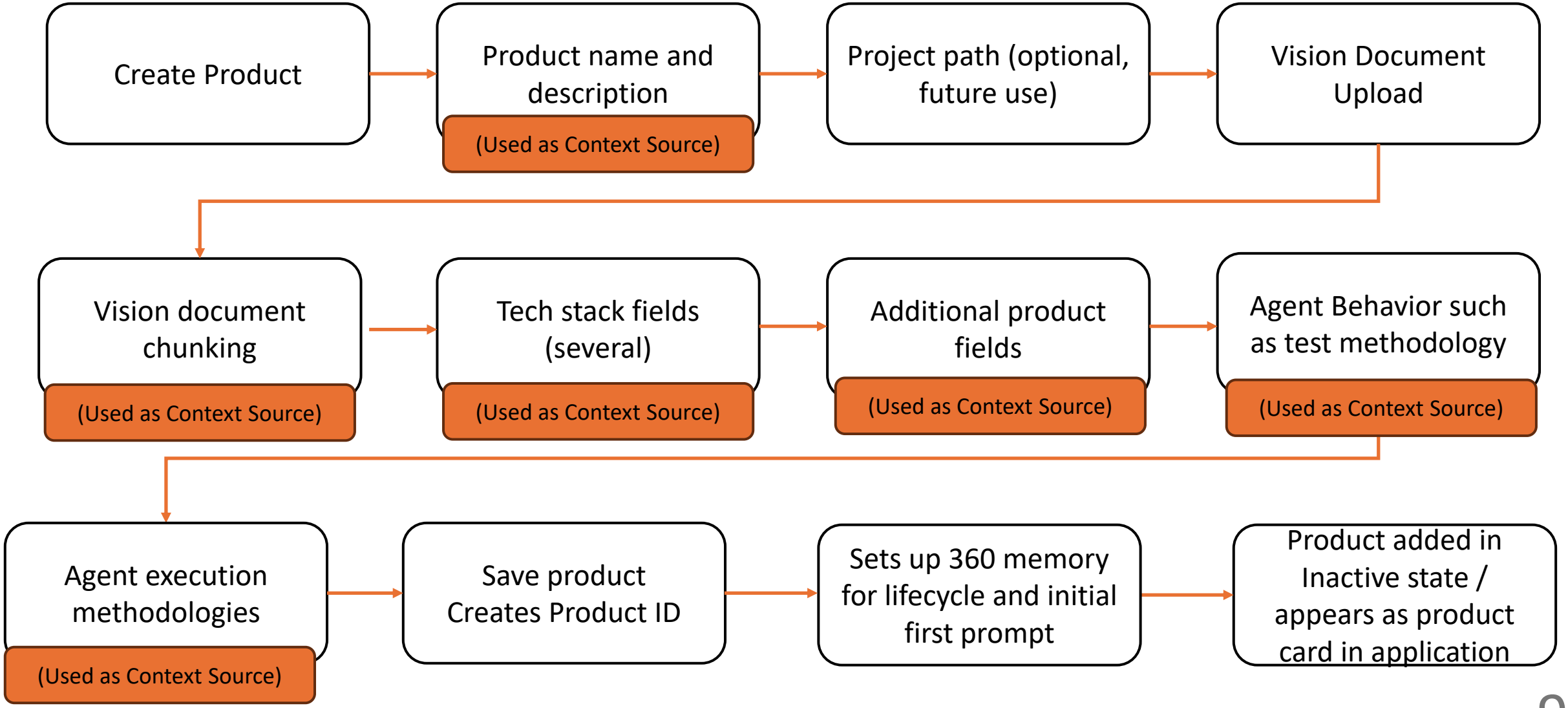
Only one product can be activated in a tenant, Only one project can be activated per product



Product Workflow overview

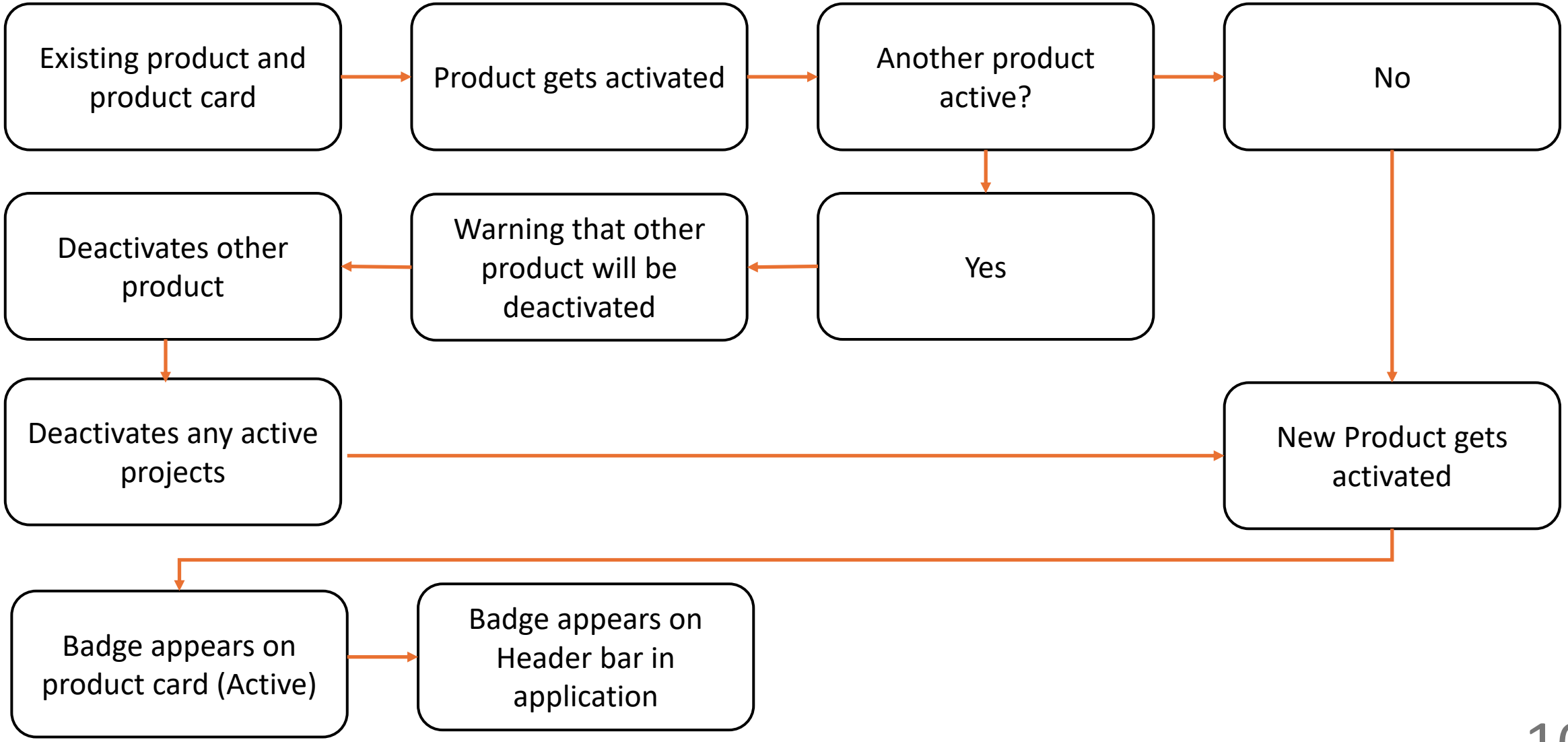
This journey is actively saved while navigated in cache during the logged in session , if a user navigates back and forth during product Add, then information repopulates

Only one product can be activated in a tennant



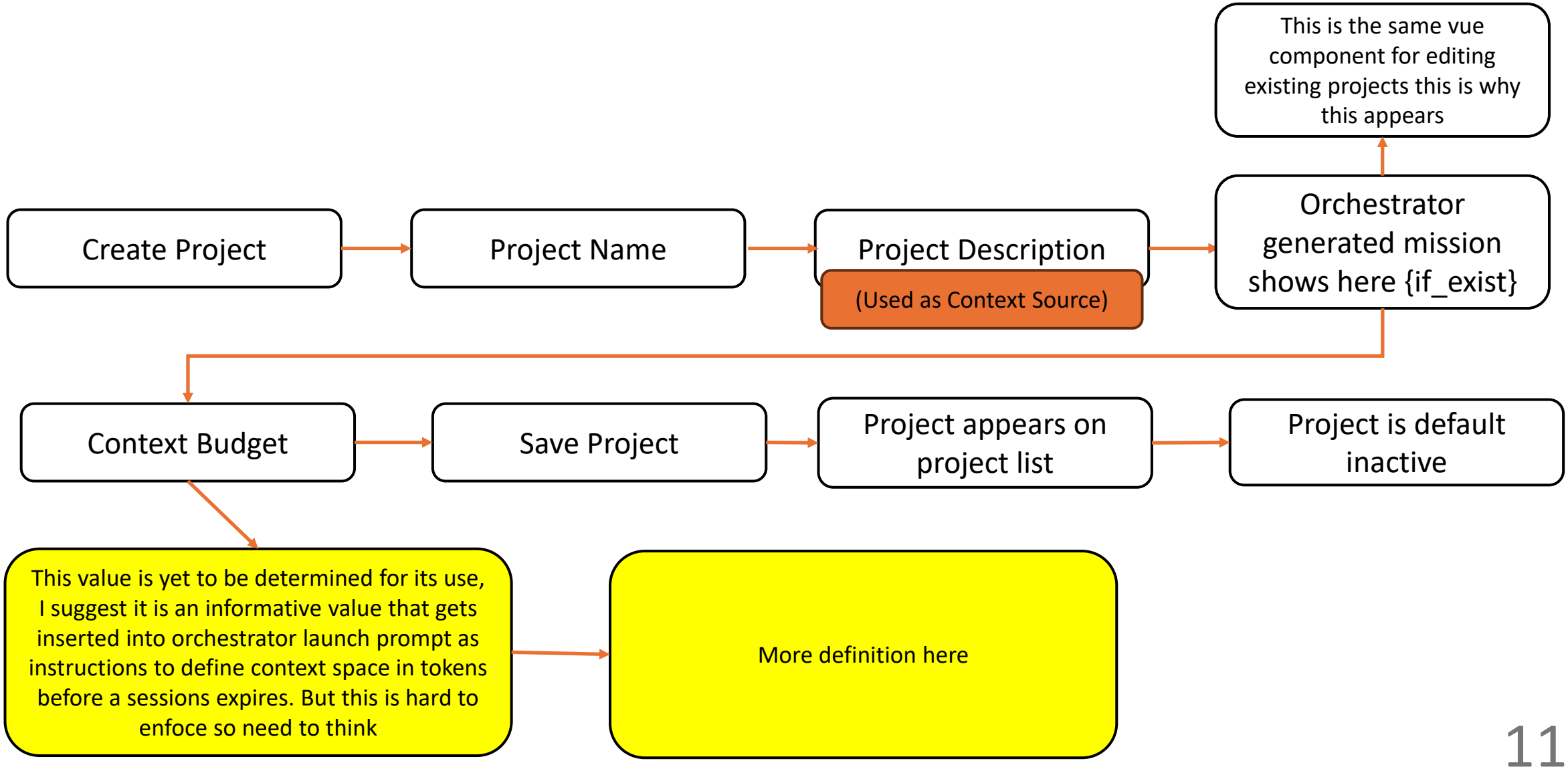
Product activation Workflow

Only one product can be activated in a tenant



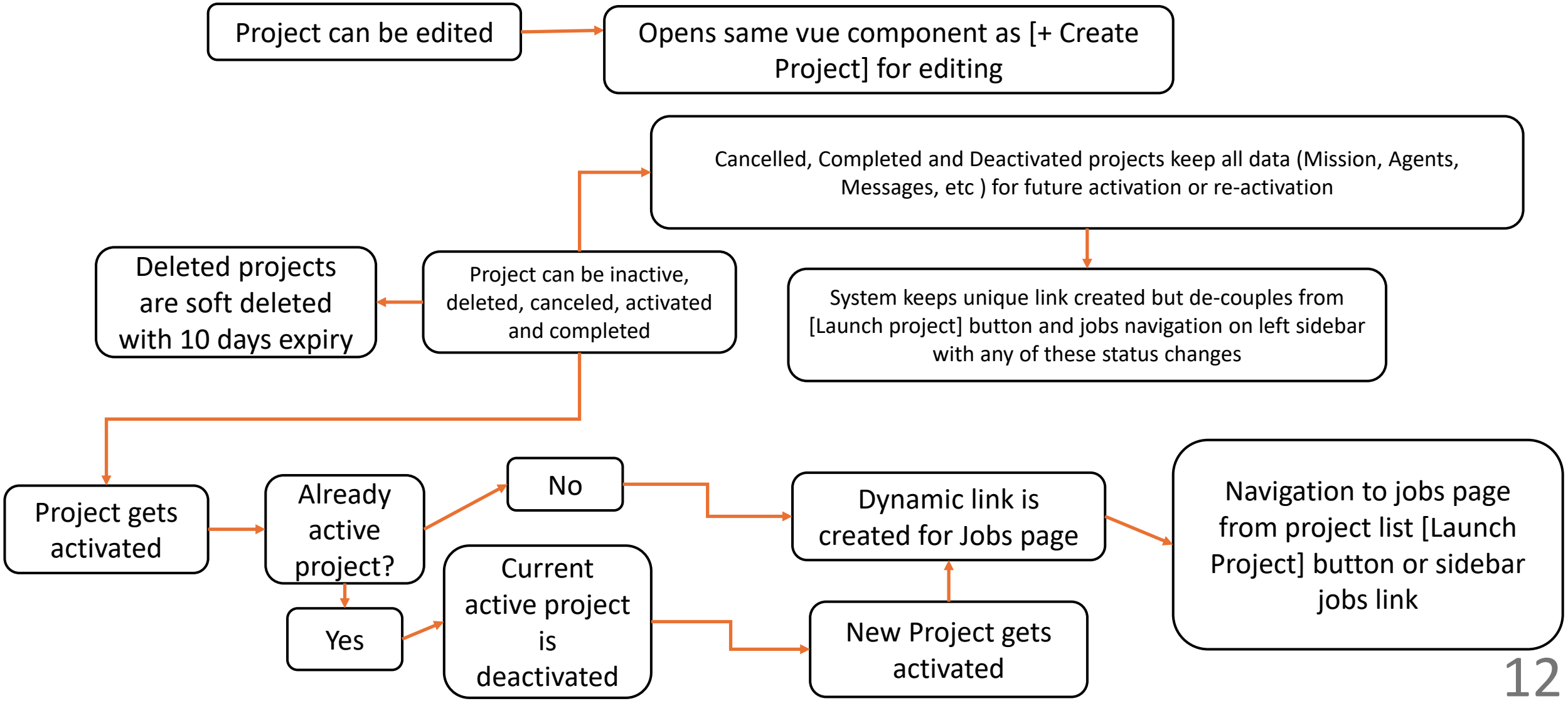
Project Creation Workflow

Only one project can be active under an activated product at any given time. Only one product can be activated in any tenant.



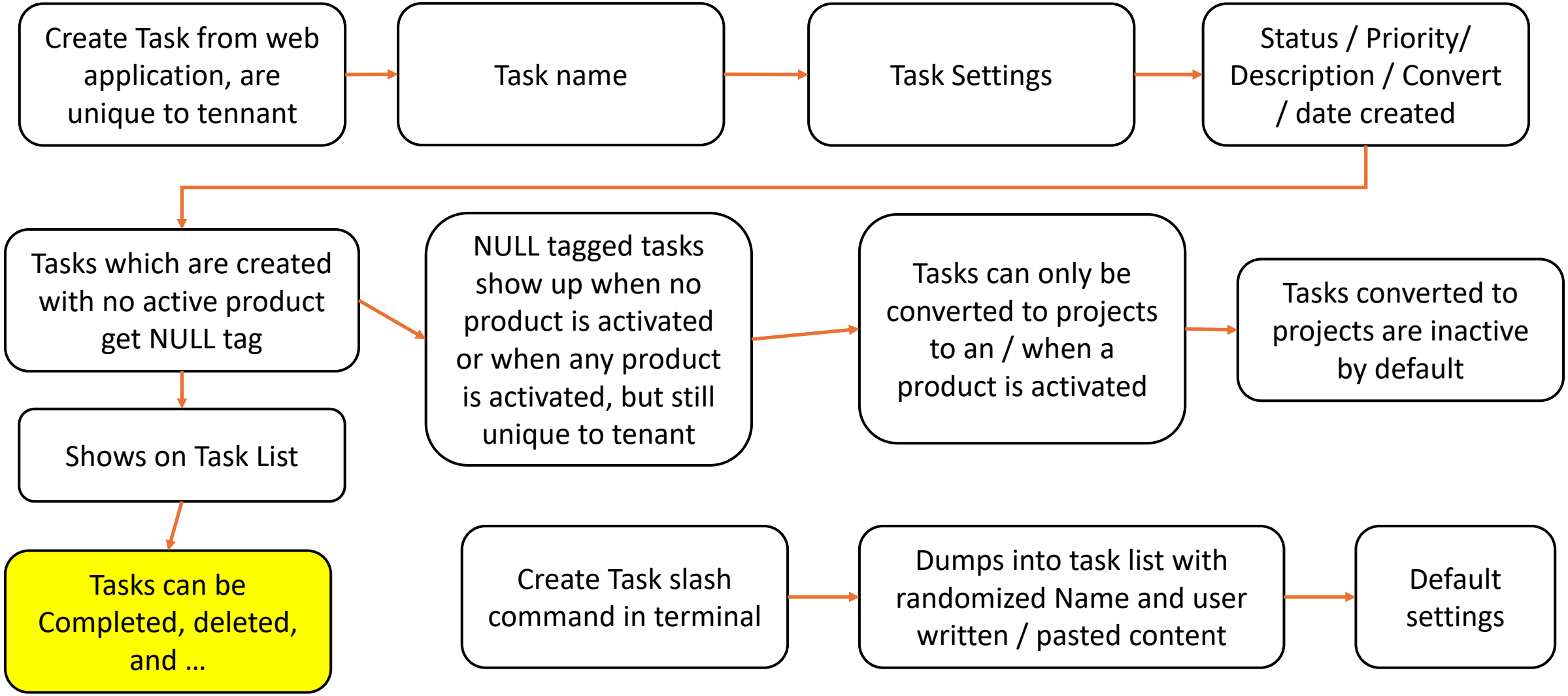
Project Modification overview

Only one project can be active under an activated product at any given time. Only one product can be activated in any tenant.



Task Workflow overview

Tasks created with active product get NULL tag and show up for all products or no products active.



Agents Workflow overview

Only 8 agent types can be active with unlimited number of agents. Agents are unique to tenants

Included Agents get seeded during application installation, but are unique to each tenant

Orchestrator Agent protected (see red box)

Default agents are enabled in Agent Template Manager

Enabled Agents have their Claude code CLI *.md templates created and staged on server

Zipped agent templates are used during Agent import function (prompt copy from web interface with tokenized {time out} link) into Claude code cli to fetch Zip and install

Enabled Agents are available for orchestrator to choose from during Project Staging in Jobs pane

(Used as Context Source)

Max 8 Agent types can be enabled, Unlimited number of each type can be activated and assigned by Orchestrator

Example, Documenter, Implementer1, Implementer2, etc.

Agents can be modified in Agent Template Manager

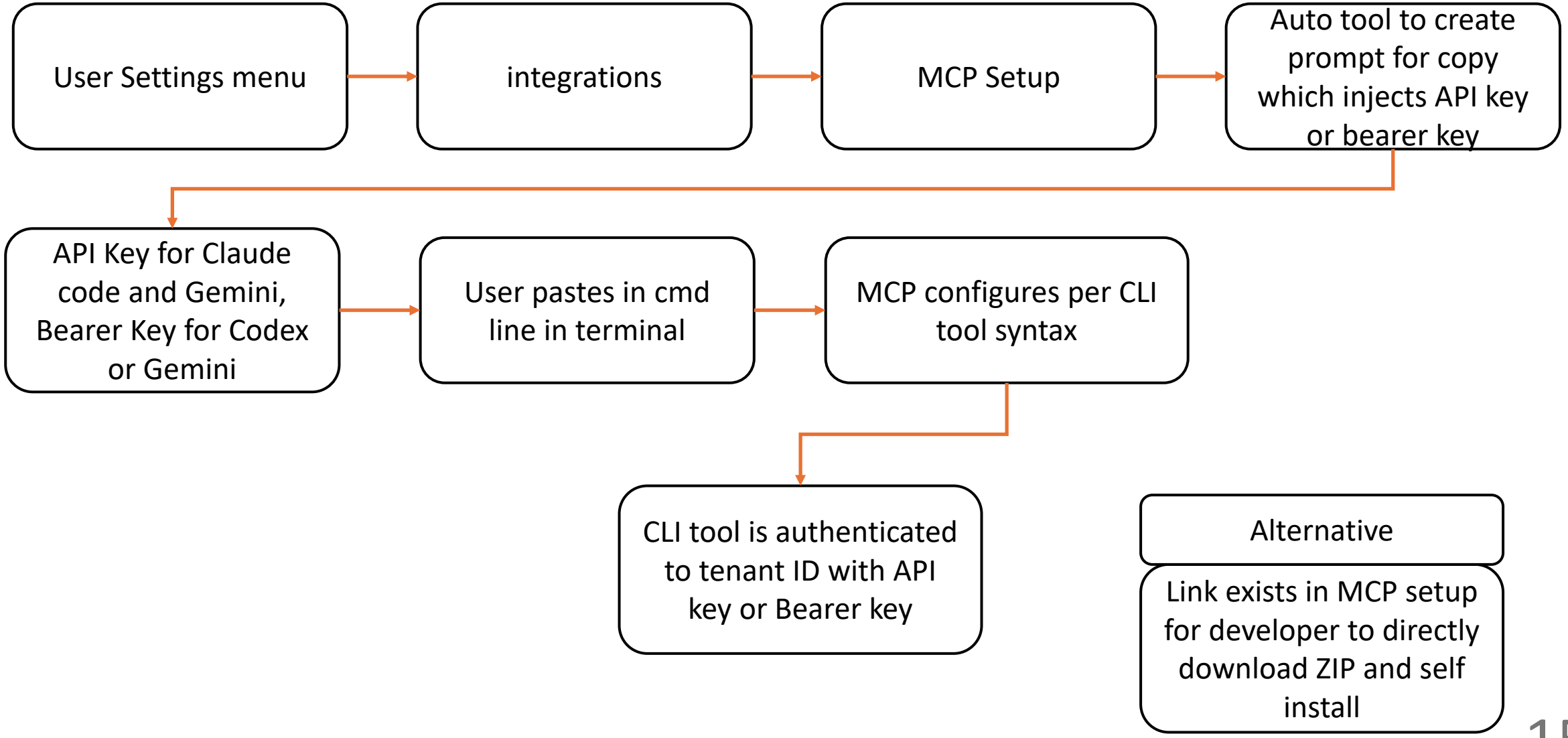
User can create their own agents if they want

Any self created agents can also be exported to Claude code CLI via export tool

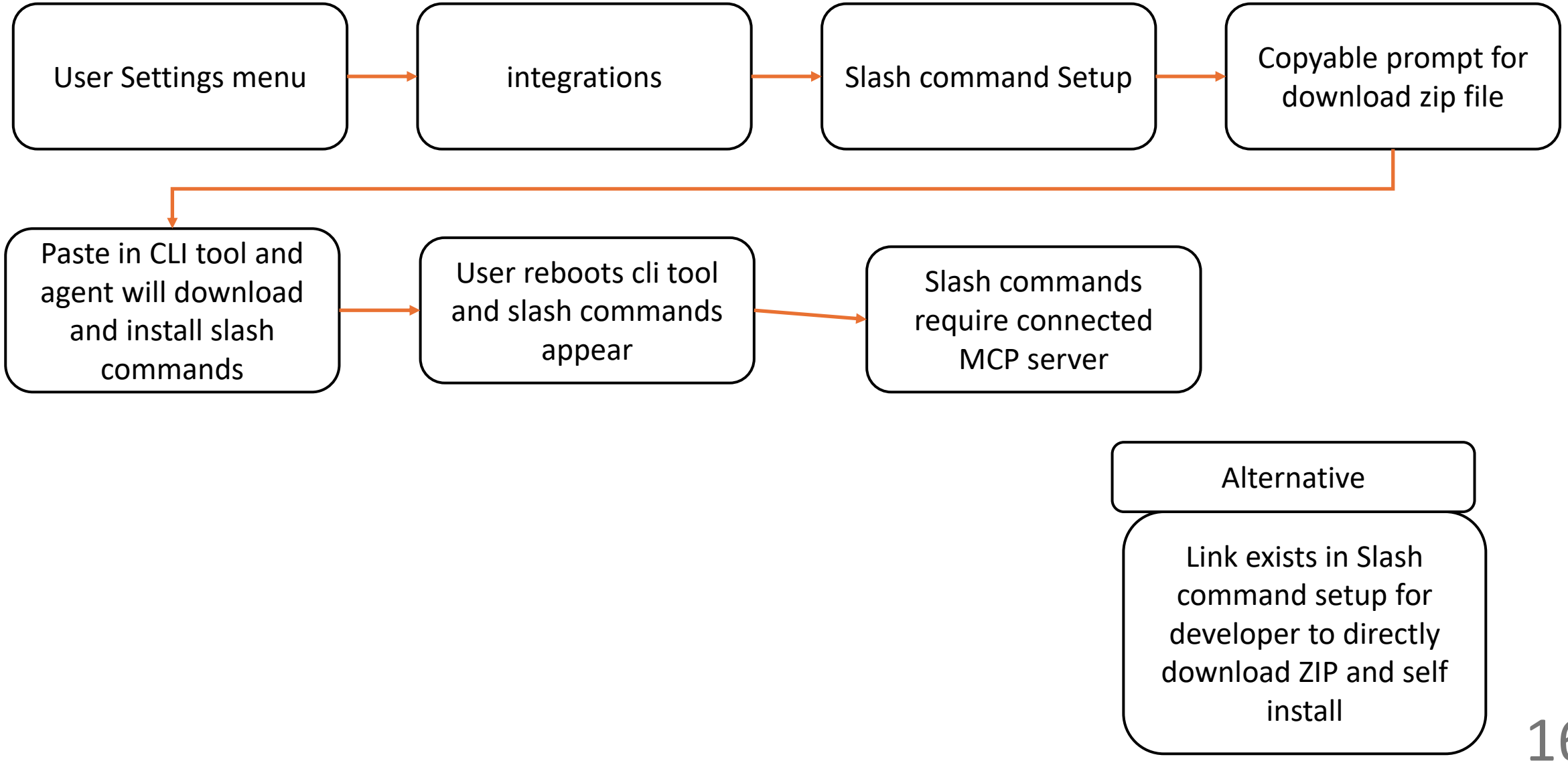
Different Agent types can be enabled up to 8

Orchestrator agent can be edit only in Admin settings with warnings

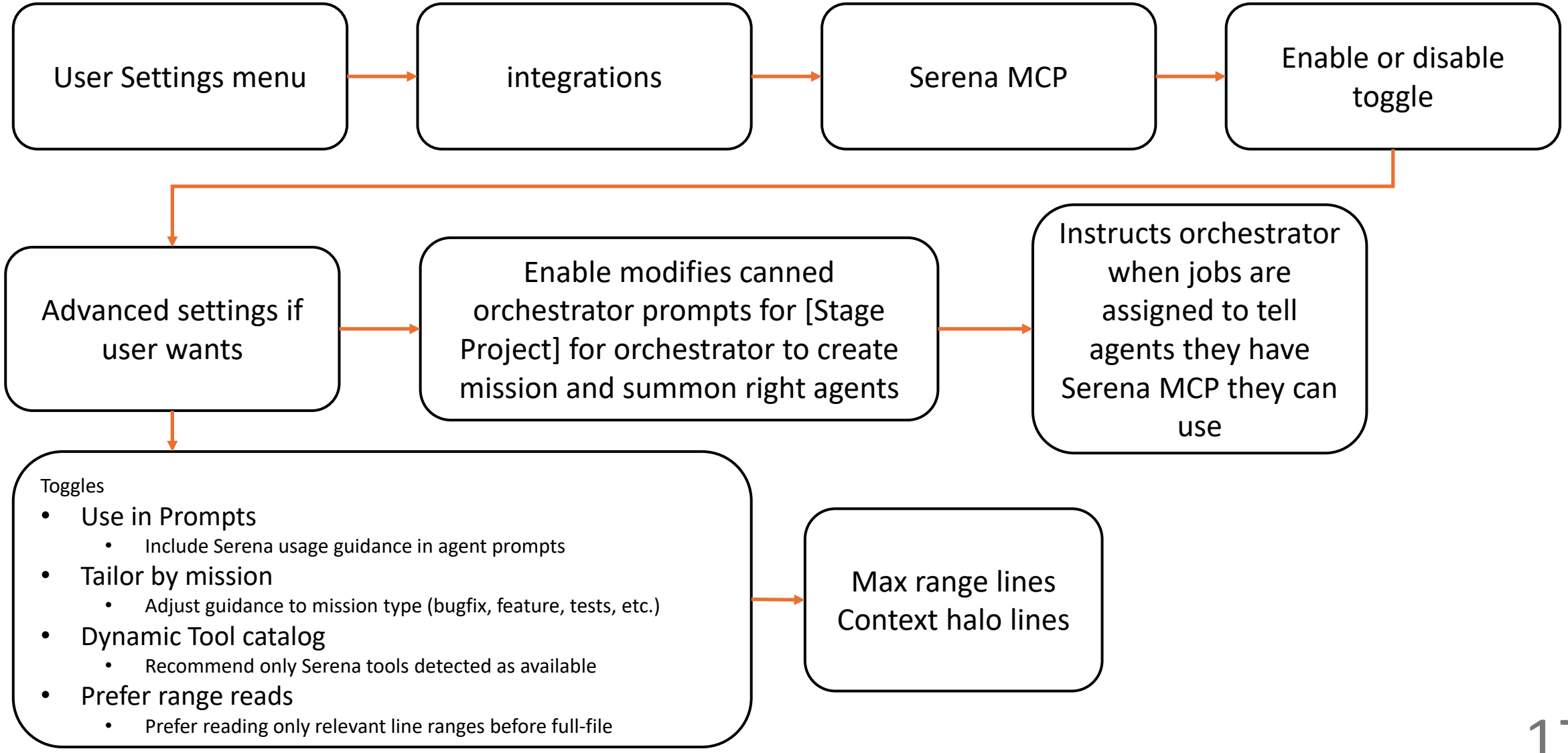
MCP to CLI conding tool setup Workflow



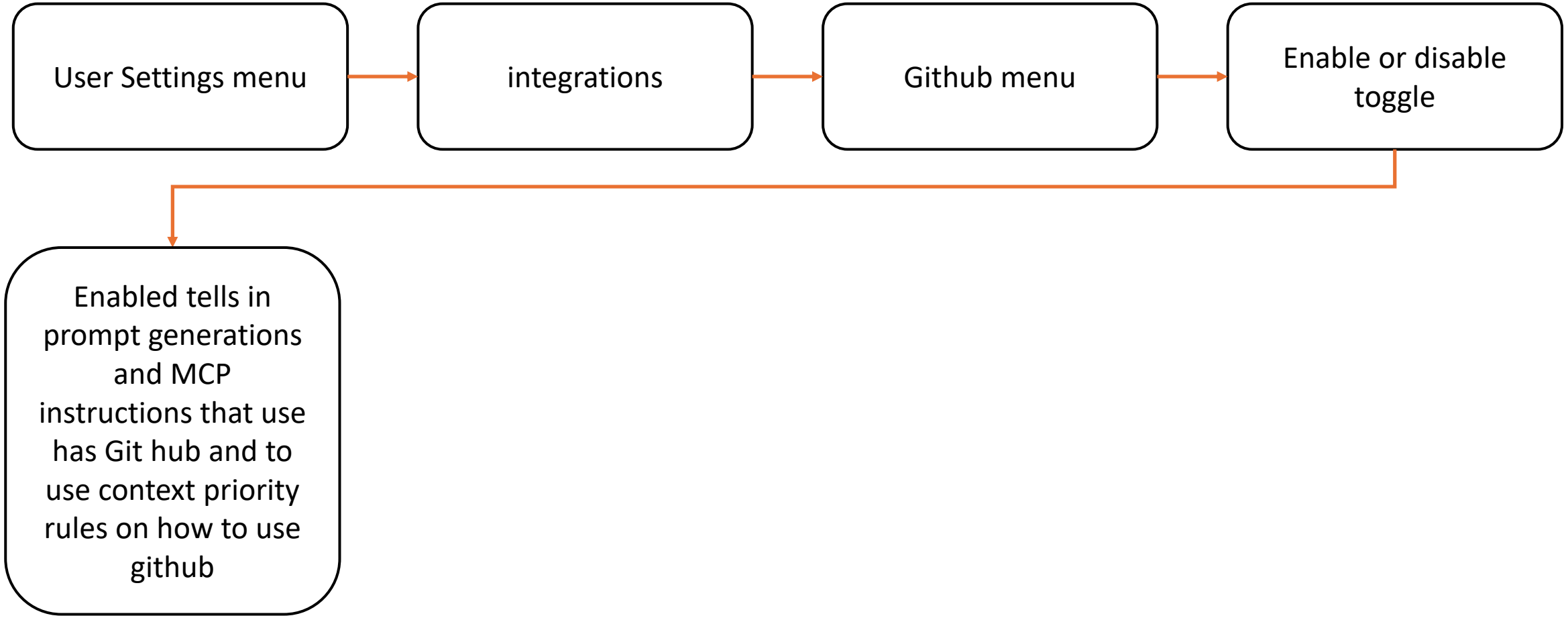
Slash command setup Workflow



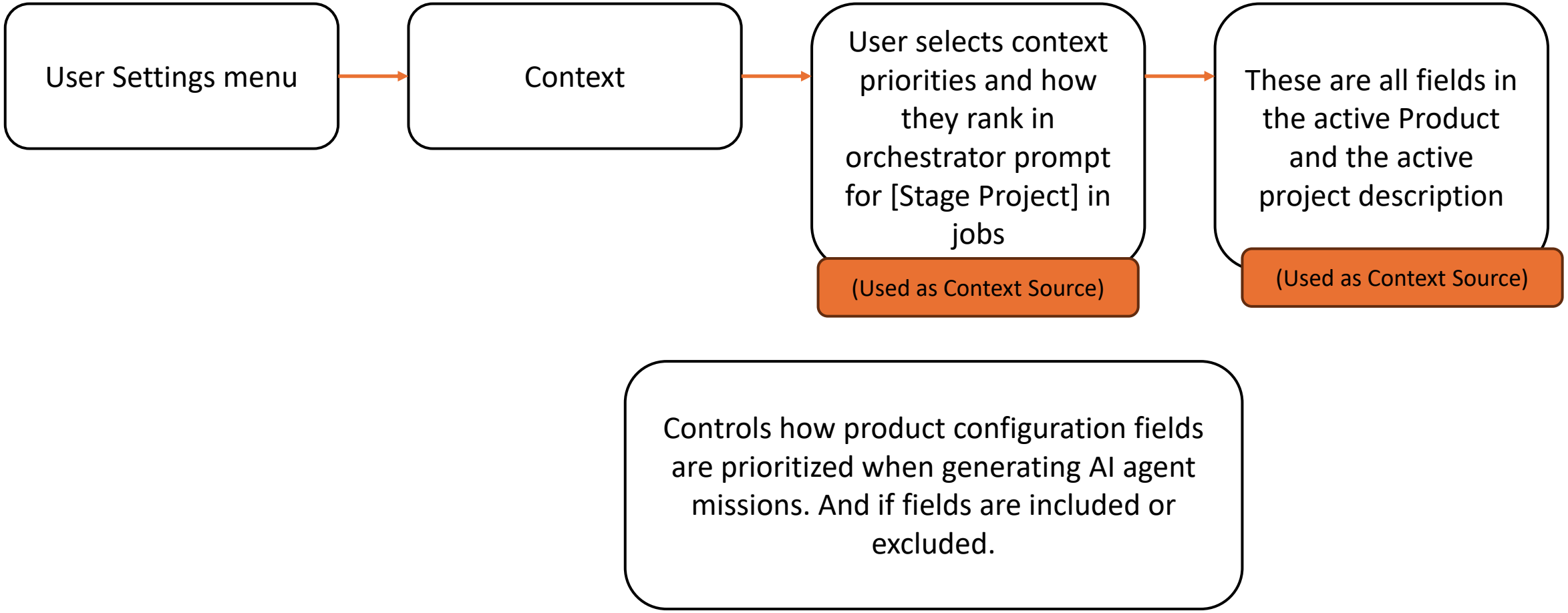
Serena MCP integration setup Workflow



Gitub integration setup Workflow



Context configuration integration setup Workflow



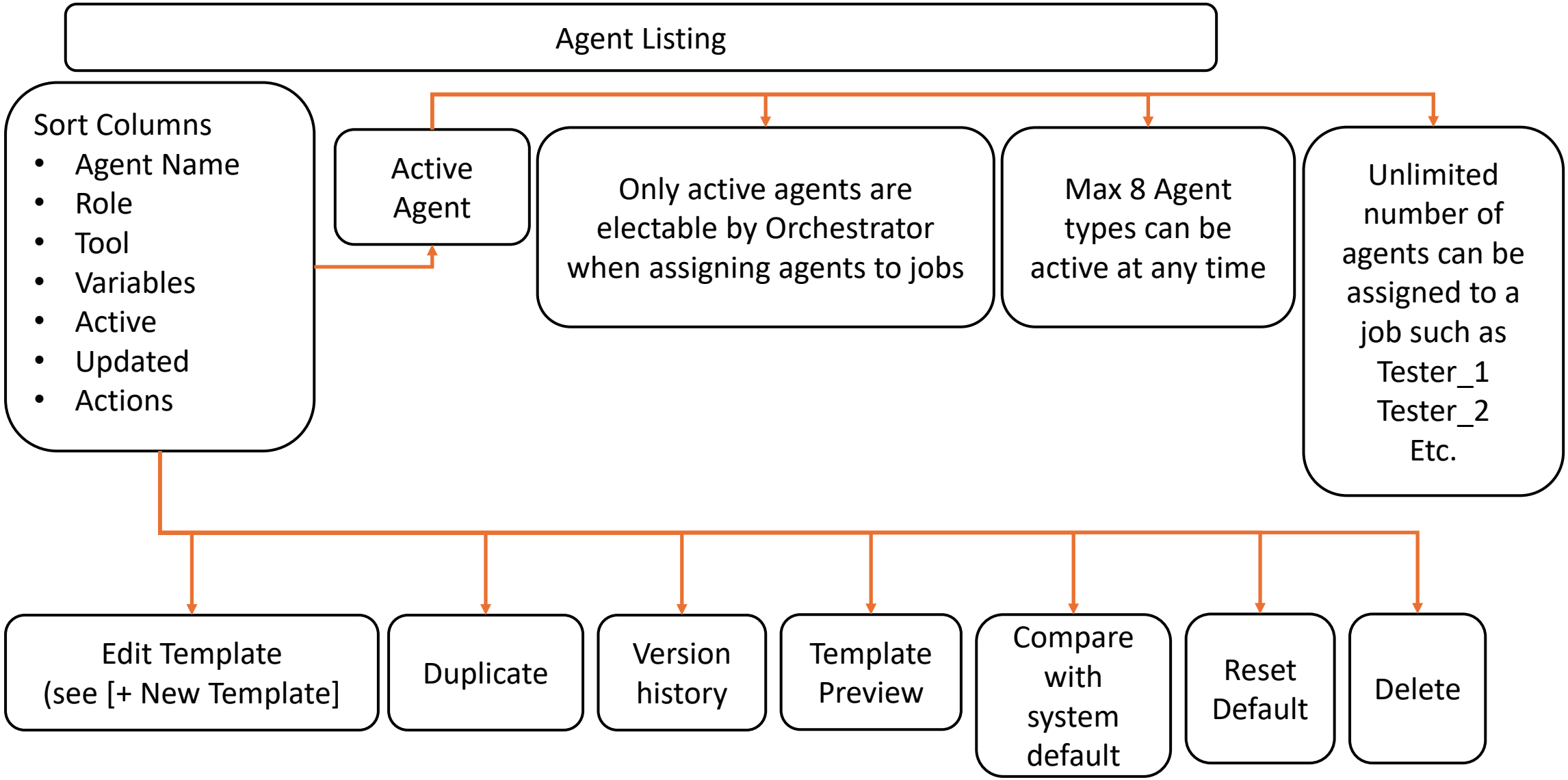
Admin Settings

- Network configuration
  - External Host IP
  - Api Port
  - Frontend Port
  - CORS origins
- Database
  - Host for intra application
  - Port
  - DB Name
  - Username / Password
  - Test connection
- Integrations
  - Lists our integrations
  - Explains how our integrations work
- Security
  - Cookie Domain Whitelist
- System
  - Orchestrator template editor
- Users?

User Settings

- Setup
- Appearance
  - Themes
  - Mascot
  - Display
  - Tooltips
  - High Contrast
- Notifications
  - Message notifications
    - New Message
    - Urgent Message
  - Agent notifications
    - Status Change
    - Agent Errors
  - Notification display location
- Context
  - User defined Context priority configuration
    (Used as Context Source)
- Agents
  - Template Manager
  - See separate flow chart
- System
  - Orchestrator template editor

Agent Manager



Orchestrator mission and job creation details

User presses ‘activates project’ on project list
Backend creates instructions job_id waiting to be fetched by orchestrator and a copiable prompt via [Stage Project] button.

Prompt contains:
Orchestrator role.
Its Agent ID
Project ID
Product ID
Fetch job_id instructions.

User copies and pastes [Stage project] prompt in terminal window to start orchestrator.

Job_id says to create Mission

Orchestrator reads fields
Product name
Product description
Product core features
Product Vision docs chunks
Product tech stack (all fields)
Product architecture (all fields)
Product testing principles (all fields)
360 memory
Git history (if enabled in user settings)

Mission is created

Mission persisted to Project.mission field via MCP tool

Orchestrator continues to see what agents are available,

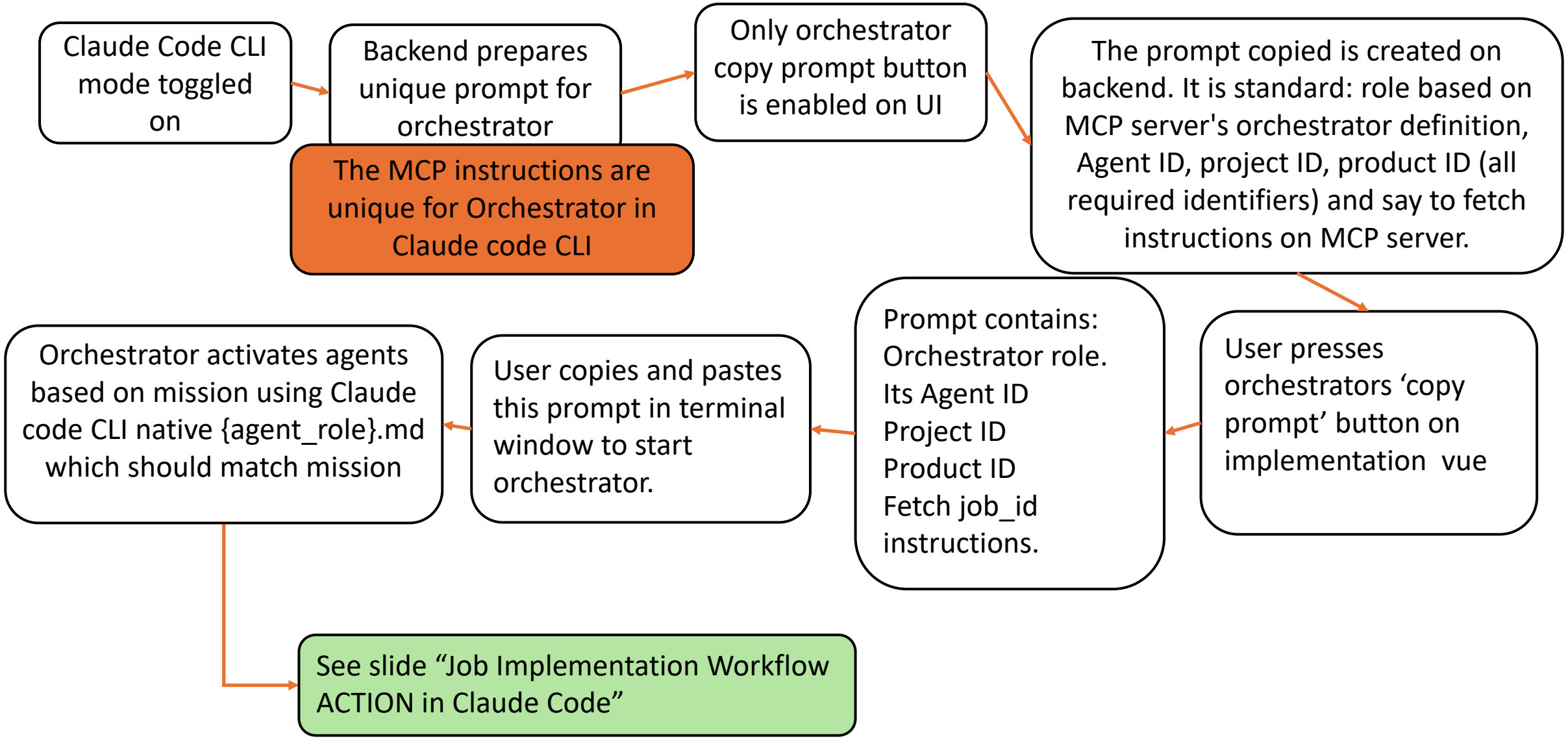
Agents are selected via ‘enabled’ toggle in agent template manager, so orchestrator does not hallucinate agent roles for the mission, and knows to which agent to break out jobs and assign job_id to

Orchestrator, assign jobs with job_id to agents and spawn them attached to the project_id Including its own new job_id during implementation phase

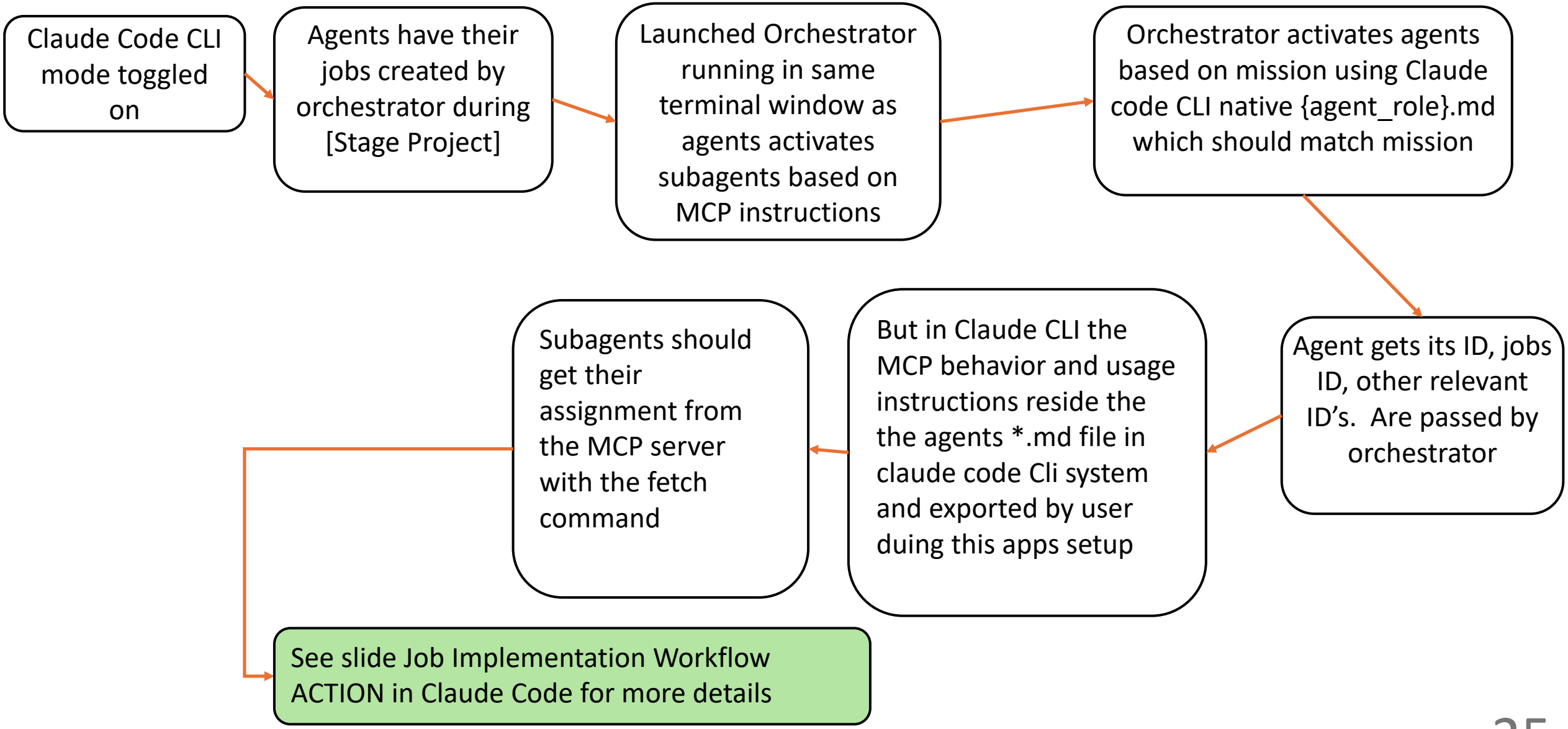
[Launch Job] button is acivated

Check slides if this is repeat or contradict

Orchestrator use during jobs in claude code CLI mode



Agent use during jobs in claude code CLI mode



Orchestrator use during jobs in multi terminal CLI mode

Claude Code CLI mode toggled OFF

Backend prepares unique prompt for orchestrator

The MCP instructions are unique for Orchestrator in multi terminal mode

All agents copy prompt buttons are enabled on UI

The prompt copied is created on backend. It is standard: role based on MCP server's orchestrator definition, Agent ID, project ID, product ID (all required identifiers) and say to fetch instructions on MCP server.

User presses orchestrators 'copy prompt' button on implementation vue

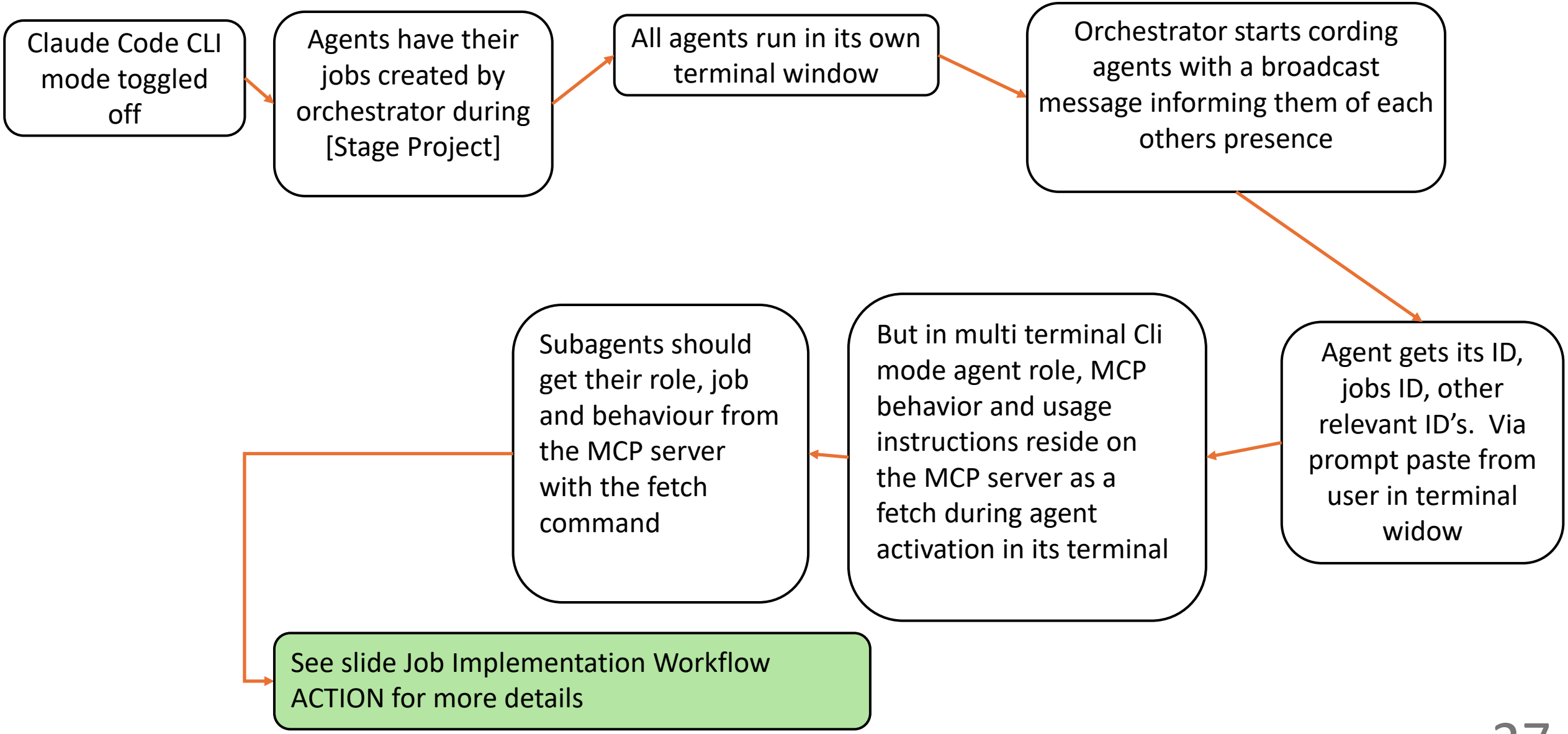
Prompt contains:
Orchestrator role.
Its Agent ID
Project ID
Product ID
Fetch job_id instructions.

User copies and pastes this prompt in terminal window to start orchestrator.

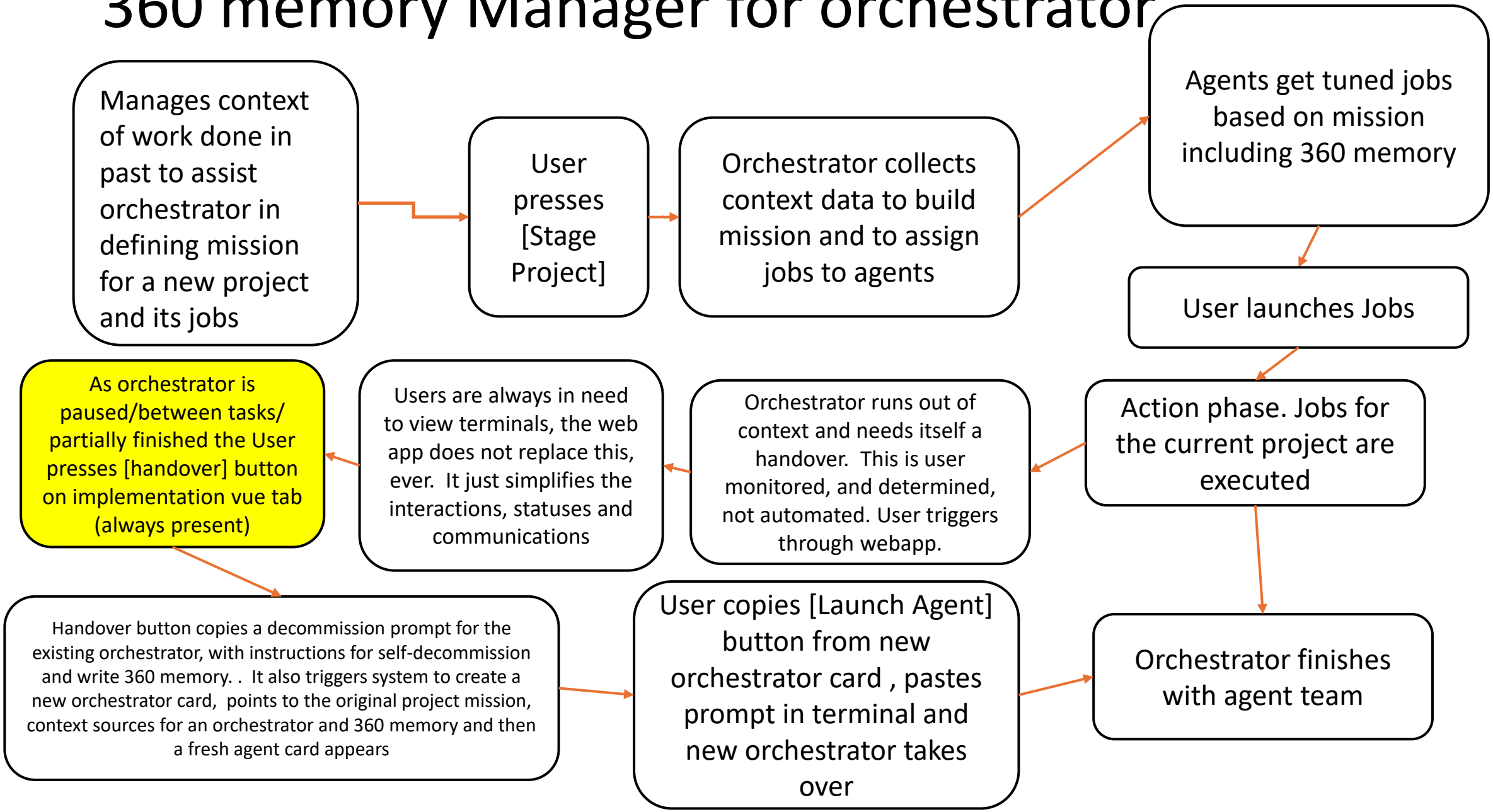
Tells orchestrator to read the mission created in staging process and inform it of all the other agents that will be spawned in unique terminal windows. Tells orchestrator to send a coordinating message to all agents and remind them to read their missions that they can send messages to other agents but note that those agents will only read those messages when active, user must manually ask them to read messages if they between instructions.

See slide Job Implementation Workflow START IN Multi Terminal mode

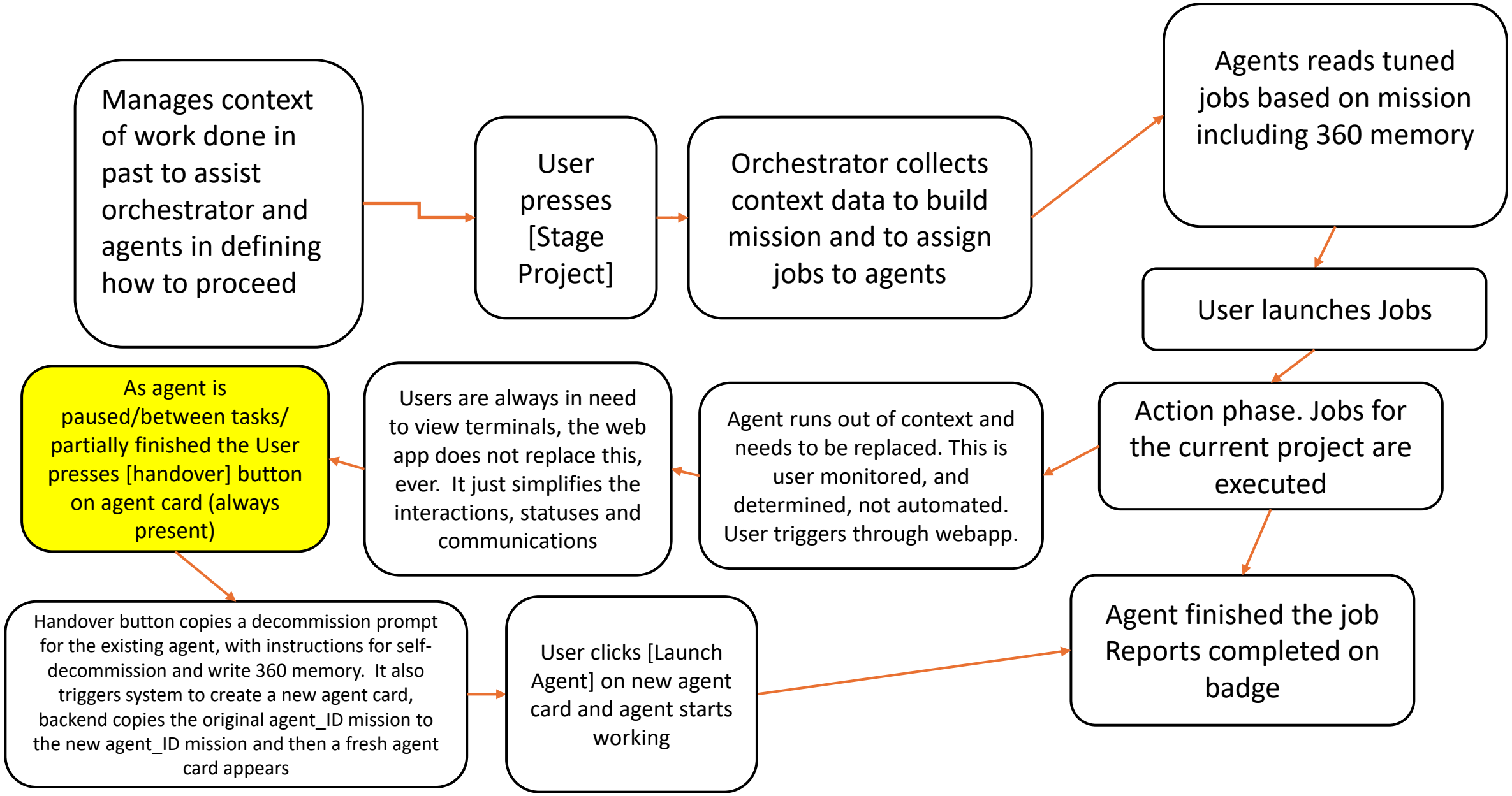
Agent use during jobs in multi terminal CLI mode



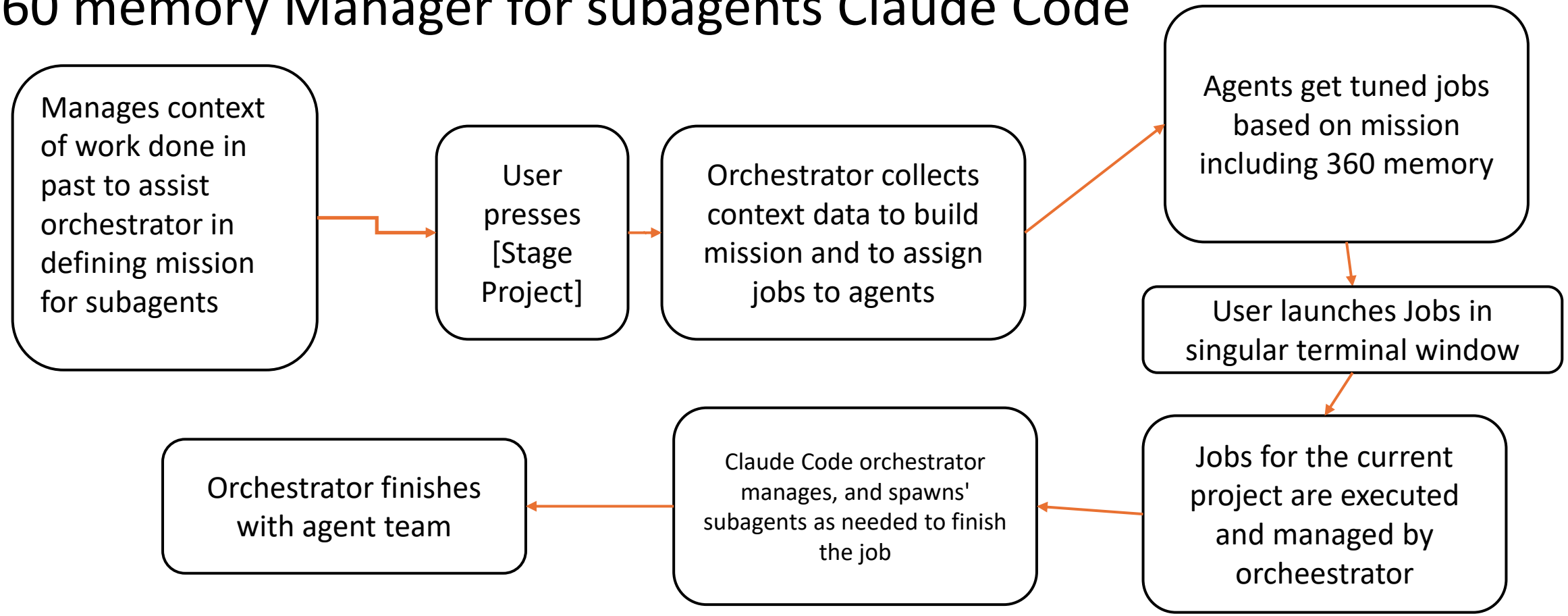
360 memory Manager for orchestrator



360 memory Manager for subagents multi terminal CLI



360 memory Manager for subagents Claude Code



Job Staging Workflow Part 1

http://{host or IP}:port/projects/{project_ID}?via=jobs vue TAB called ‘Launch’

Project is Activated

Unique product link is created

http://
{host or IP}:port/projects/{project_ID}?via=jobs

Reachable via [Launch Project] button in project list or Jobs left side nav

Page appears with [Stage Project] Button Startin in vue TAB ‘Launch’

The project description appear and is editable

An empty “Orchestrator Generated Mission” window appears

Initial Agent card for the system specified Orchestrator Agent appears with {agent_ID}

This Orchestrator role definition is injected into the copyable prompt

See Job Staging Workflow Part 2, slide 27

Job Staging Workflow Part 2

Uses presses [Stage Project] Button from http://{host or IP}:port/projects/{project_ID}?via=jobs

Button creates prompt for copy and paste into a terminal running Agentic AI coding tool so it can stage the job for the project

User copies and pastes Prompt

Agentic CLI tool becomes "Orchestrator" Agent and reads instructions on MCP server

MCP instructions say to read contexts and sources material available via MCP commands

MCP instruction tells Orchestrator if Serena or Git hub can be used, enabled or disabled in user settings

Context fields

Product name
Product description
Product core features
Product Vision docs chunks
Product tech stack (all fields)
Product architecture (all fields)
Product testing principles (all fields)
360 memory
Git history (if enabled in user settings)

Orchestrator checks what subagents templates it can use based on template manager in user settings.

Orchestrator creates mission based on all context

Mission appears on the Jobs window once written

Agent cards appear with {agent_ID} and their created jobs

[Launch Jobs] Button appears, a nav link to new vue TAB in jobs link

Job Implementation Workflow START IN CLAUDE Code CLI Mode

http://{host or IP}:port/projects/{project_ID}?via=jobs vue TAB called ‘Implementation’

User clicks dynamic button [Launch Jobs] after orchestrator finishes mission and agent assignments

Moves to implementation window

Claude Code CLI mode toggled on

Only orchestrator copy prompt button is enabled

The prompt copied is created on backend. It is standard: role based on MCP server's orchestrator definition, Agent ID, project ID, product ID (all required identifiers) and say to fetch instructions on MCP server.

The MCP instructions are unique for Orchestrator in Claude code CLI

Tells orchestrator to read the mission created in staging process and to spawn agents using native Claude Code CLI subagent capability. Providing each subagent its ID, other relevant ID’s and job ID to fetch on MCP server. Orchestrator can choose to spawn agents in parallel or sequential mode based on its own reasoning

User copies and pastes prompt in Claude code Cli terminal window

Goto slide : Job Implementation Workflow ACTION in Claude Code slide

Job Implementation Workflow ACTION in Claude Code

http://{host or IP}:port/projects/{project_ID}?via=jobs vue TAB called 'Implementation'

User has started orchestrator in Claude Code CLI terminal

Orchestrator reads its role definition, gets all ID's needed and fetches job on MCP server. It reads the mission first. Then its instructions.

Orchestrator activates agents based on mission using Claude code CLI native {agent_role}.md which should match mission

Orchestrator in its own MCP instructions is told to pass, during agent spawn include each agents ID, job ID all unique ID's and tell them their Job is waiting

Our application has agent template export during initial setup, so now the {agent}.md templates should contain MCp instructions for the agent

The agents aware if its ID' all relevant ID's and the job ID fetches its instructions. Executes job

When all agents are complete user can after reviews , tests, validation close out the project by copy pasting 'close out prompt'

User can keep instruction orchestrator either in terminal or via message center in application

Project gets tagged as completed and agents are decommissioned. Cannot be MCP interacted with, nor can the project be modified from outside during other projects

Project is readable, shows in history of dashboard, can be reactivated if needed, reactivating agents

Job Implementation Workflow START IN Multi Terminal mode

http://{host or IP}:port/projects/{project_ID}?via=jobs vue TAB called 'Implementation'

User clicks dynamic button [Launch Jobs] after orchestrator finishes mission and agent assignments

Moves to implementation window

Claude Code CLI mode toggled OFF

All agents cards have active copy prompt button

All agent promptsare created on backend. It is standard: role based on MCP server's agent type definition, Agent ID, project ID, product ID (all required identifiers) and say to fetch instructions on MCP server.

The MCP instructions are unique for orchestrator in Multi Terminal mode

Tells orchestrator to read the mission created in staging process and inform it of all the other agents that will be spawned in unique terminal windows. Tells orchestrator to send a coordinating message to all agents and remind them to read their missions that they can send messages to other agents but note that those agents will only read those messages when active, user must manually ask them to read messages if they between instructions.

All other agents prompts say to read the agents unique job created in staging process & informs it of all the other agents that will be spawned in unique terminal windows. Tells to read any messages & that they can send messages to other agents but note that those agents will only read those messages when active, user must manually ask them to read messages if they between instructions.

All prompts are copied and pasted in unique terminal windows

Goto slide : Job Implementation Workflow ACTION in multi terminal mode

Job Implementation Workflow ACTION in Multi terminal mode

http://{host or IP}:port/projects/{project_ID}?via=jobs vue TAB called 'Implementation'

User has started orchestrator in its own terminal

Orchestrator reads its role definition, gets all ID's needed and fetches job on MCP server. It reads the mission first. Then its instructions.

Orchestrator sends a coordinating message to all agent via MCp message system informing them of all other agents working and their ID's

All other agents are started by user via copy paste in their own terminal widows

Agent reads its role definition, gets all ID's needed and fetches job on MCP server. Checks messages, gains awareness of all other agents working as a team

The agents aware if its ID' all relevant ID's and the job ID fetches its instructions. Executes job

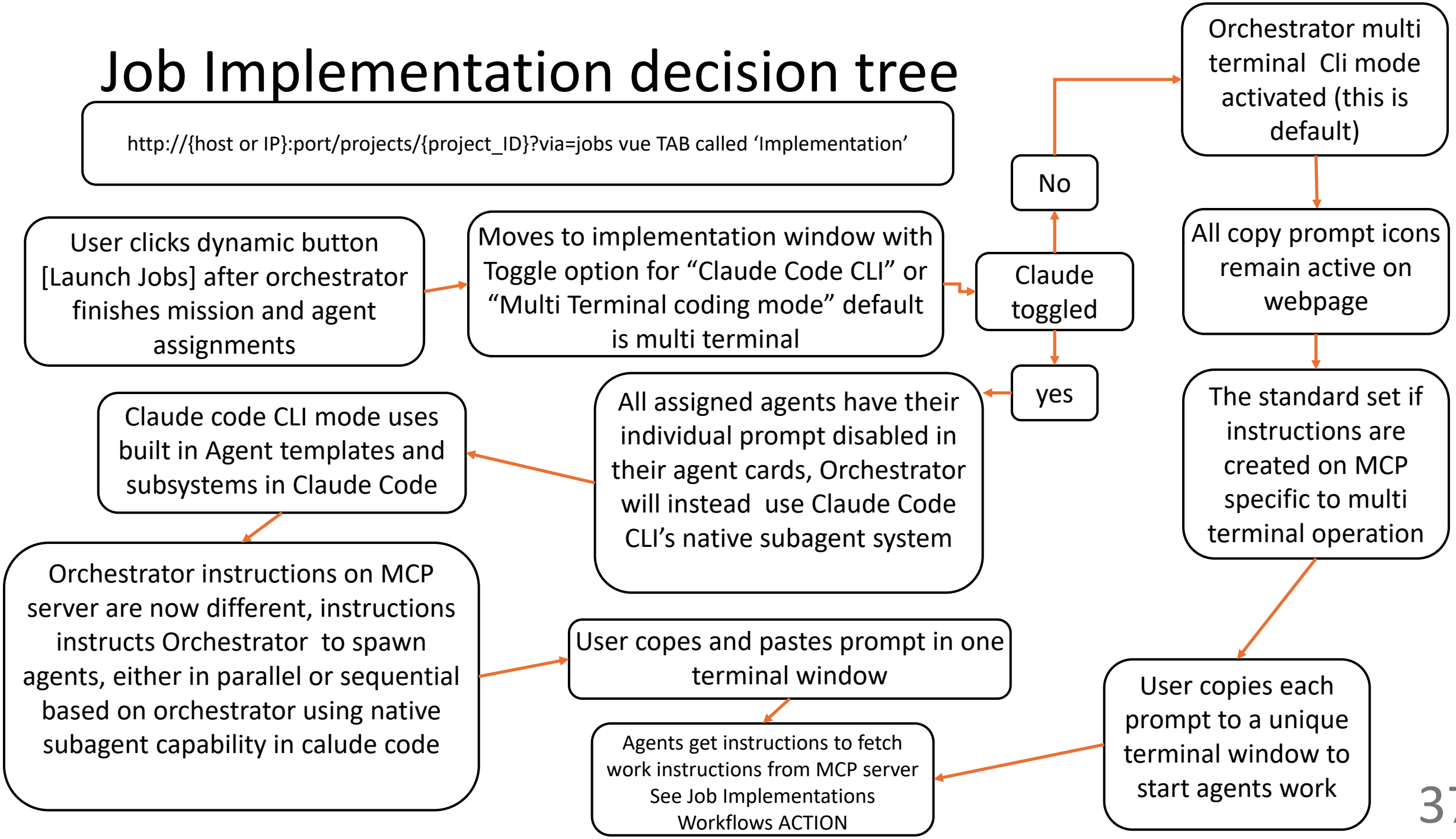
When all agents are complete user can after reviews , tests, validation close out the project by copy pasting 'close out prompt'

User can keep instructing orchestrator either in terminal or via message center in application

Project gets tagged as completed and agents are decommissioned. Cannot be MCP interacted with, nor can the project be modified from outside during other projects

Project is readable, shows in history of dashboard, can be reactivated if needed, reactivating agents

Job Implementation decision tree



Job Implementation Workflow ACTION

http://{host or IP}:port/projects/{project_ID}?via=jobs vue TAB called ‘Implementation’

In claude code mode this is native using {agent_role}.md

Agents are working

Agent templates and job instructions enforce the need for MCP communications as they work

Agents communicate via MCP to update status , visible in a message pane

User can send messages to agent via message pane. Orchestrator direct or Broadcast to all agents.

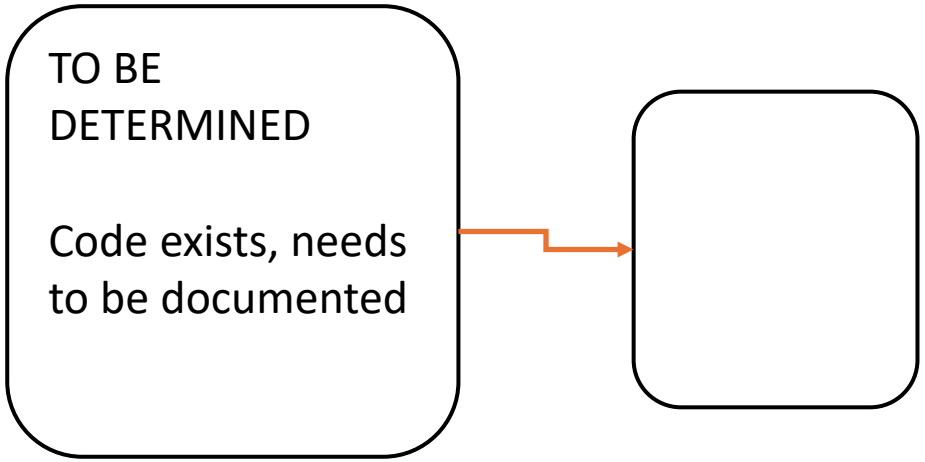
Messages become part of auditable functions (future), currently the user is just informed

User can at any time interact via CLI tool terminal windows and often will, as it bypasses check ins and queues

When all agents are complete user can after reviews , tests, validation close out the project

User can keep instruction agents and change their status to keep going if needed

Project gets tagged as completed and agents are decommissioned. Cannot be MCP interacted with, nor can the project be modified from outside during other projects

Project is readable, shows in history of dashboard, can be reactivated if needed, reactivating agents

Decommissioning